

Slot-Chain: A Preconfirmation Protocol

Production Design Candidate — v2.2

August 2026

Abstract

Slot-Chain assigns one-second L2 slots to bonded builders. Builders sign parent-linked blocks off chain; any party can prove and submit a batch to L1. Normal candidates compete for a fixed settlement interval. After an objective finality breach, or when a forced L1 message becomes due, anyone can prove an unsigned deterministic escape block. Recovery is an episode of renewable, objectively expiring rounds, so a prover outage cannot make its target stale forever. This revision also fixes schedule sealing, candidate anchors, forced resource bounds and ordering, immutable data sessions, the verifier ABI and atomic migration. It is an implementation candidate, not a production authorization: the proving, gas, economics and adversarial gates in §13 remain mandatory.

Contents

1	Status, claims and exclusions	3
1.1	Claims	3
1.2	Non-claims	3
2	Actors and trust model	3
3	Clock domains and canonical state	4
4	Builder registry and exact lookahead	5
4.1	Registry lifecycle	5
4.2	Authenticated snapshot	5
4.3	Consensus byte encoding and allocation	6
5	Blocks, promises and proof statement	6
5.1	Signed header	6
5.2	Three validity tiers	7
5.3	Verifier statement ABI	8
5.4	Preconfirmation semantics	8
6	Blob data authentication	9
7	Settlement and recovery state machine	10

7.1	Normal candidates	10
7.2	Activation	10
7.3	Recovery acceptance	11
8	Forced queue and unsigned escape	11
9	Economics, slashing and payments	13
10	Security and liveness analysis	13
11	Implementation blueprint	14
11.1	L1 modules and bounded storage	14
11.2	Historical authentication	15
11.3	Reorg rule	15
11.4	Genesis and migration	15
12	Parameters and enforced geometry	15
13	Production gates and verdict	16
A	Normative commitment encodings	18
B	Normative transition ordering	19
C	Rejected alternatives	20
D	Executable consensus models	20
E	Security invariants checklist	21

1 Status, claims and exclusions

Normative words **MUST**, **MUST NOT** and **SHOULD** have their usual RFC meaning. Where prose and either executable model disagree, deployment is blocked until the disagreement is resolved; neither source silently overrides the other.

1.1 Claims

Subject to L1 safety/liveness and a sound validity-proof system, the protocol claims:

1. **Finalized-state safety.** L1 accepts only a proved state transition extending the stored canonical tuple. Builder signatures never substitute for execution validity.
2. **Bounded protocol recovery.** A builder cartel, an absent aggregator, an empty builder registry, and missing standbys cannot prevent an unsigned escape candidate from advancing the canonical state.
3. **Forced transaction inclusion.** A correctly prepaid L1 envelope at the queue head is consumed by the next canonical block after it becomes due; it is executed if valid and consumed-and-discarded if invalid.
4. **Permissionless landing and recovery.** No allowlist, builder key, aggregator seat or payment recipient is required to submit a proof or an escape candidate.
5. **Deterministic lookahead.** Every implementation derives the same schedule from an authenticated finalized snapshot and the byte encoding in §4.

1.2 Non-claims

- A preconfirmation is not finality or insurance. Before normal close it may be revoked by builder equivocation, an unprovable skip, a withheld body, or an escape commit. Applications must price this as probabilistic/economic assurance and publish their own exposure limit.
- A bond cannot cap aggregate off-chain promises: the protocol has no reservation ledger and a builder can sign mutually exclusive promises. L_LEASE is deterrence and a recovery source, not guaranteed coverage.
- Protocol liveness is a capability claim. Economic liveness additionally assumes at least one actor can fund proof generation and an L1 transaction. A payment failure never invalidates an otherwise valid canonical transition.
- Censorship by L1 for longer than T_INCLUDE_MAX, failure of L1 finality, or a broken proof system is outside the model.

2 Actors and trust model

Builder.

A registered address eligible for scheduled slots. Builders may equivocate, skip parents, withhold bodies, collude, or disappear.

Aggregator.

The current auction seat, expected to collect data, prove and submit normal candidates. It has no exclusive consensus authority.

Prover/lander.

Any address. It supplies a validity proof and names an optional reward beneficiary in the proof statement. Proof validity, not identity, authorizes transition.

Forced-message sender.

Any L1 account that submits a gas-bounded, prepaid L2 transaction envelope.

L1.

Ethereum provides canonical ordering, timestamps, EIP-4788 beacon roots, EIP-2935 historical block hashes, blob commitments and the KZG point-evaluation precompile.

The safety threshold contains no honest-builder assumption. Honest builders improve normal throughput and preconfirmation reliability; the unsigned escape lane provides state and forced-queue progress when all of them collude. The L1-liveness assumption includes transaction inclusion and at least 64 canonical L1 blocks becoming available within $T_DEPTH_MAX=900s$; a longer L1 stall suspends, rather than violates, the L2 recovery clock.

3 Clock domains and canonical state

Before activation, Settlement is in `PREAMBLE`; candidate submission is disabled and slot arithmetic saturates. L2 slot zero is fixed by `GENESIS_TIMESTAMP`, with

$$\text{currentL2Slot} = \max(0, \text{block.timestamp} - \text{GENESIS_TIMESTAMP}).$$

L2 slot and every deadline are timestamp seconds. L1 confirmation depth is counted in L1 block numbers. Beacon slot is used only in schedule authentication. Implementations **MUST** NOT substitute one clock for another.

The L1 contract stores the following canonical tuple:

```
Canonical = (tipHash, tipSlot, stateRoot, messageCursor,
             canonicalizationBlock, winningDataCommitment)
```

`canonicalizationBlock` is the L1 block that most recently committed the tuple; no code attempts to read or store `blockhash(block.number)`, which does not exist during that block. Candidate acceptance compares the proved base-tuple hash directly with current contract storage. A candidate has one prior-block anchor. Normal anchors are authenticated by an EIP-2935 block hash, the supplied execution header and MPT proofs. A recovery round instead stores `blockhash(block.number-1)` while available and requires that anchor to reach depth `F_L1`. An L1 reorg rolls the stored tuple and anchor back together. Initial values and bridge cursors are migration inputs in §11.

Every state-mutating external entry point begins with `sync()`. If `sync()` changes mode or commits a mature normal candidate, the wrapper **MUST** return `SYNCED` successfully before parsing or validating the caller's candidate. The caller resubmits against the new version. This rule is necessary because a later Solidity revert would otherwise roll back the `sync` transition in the same transaction.

4 Builder registry and exact lookahead

4.1 Registry lifecycle

A registration contains a nonzero address, a uint192 bond not exceeding `BOND_MAX`, a monotonic uint64 `registrationIndex`, and separately reserved `L_LEASE[window]` tranches. Address zero is permanently reserved for `VACANT`. Duplicate live addresses and re-registration of a tombstoned key are forbidden.

The active table has exactly 64 indexed cells. Admission into a full table atomically replaces only its minimum-bond live entry after the delayed-entry rule; displaced entries retain historical liabilities. Because the complete table is bounded and authenticated, snapshot verification proves completeness rather than accepting an unverifiable claimed “top 64.” Every registry mutation updates a fixed-depth Keccak `registryRoot` over all 64 cells.

Eligibility is evaluated before ranking: an entry is eligible for window W only if its registration was effective in the authenticated snapshot, it had a fully reserved tranche for W , and its tombstone was not effective at that snapshot. The 64 eligible entries with greatest bond, then smallest registration index, form the snapshot set. If none exist, every ticket is `VACANT`. A later slash tombstones the key and increments `admissionVersion`. An open normal settlement window freezes the prior version, so a slash cannot invalidate its accepted best; the new tombstone applies to later normal windows and every recovery round. Client lookahead renders tickets at or after the effective slot as `VACANT`, while clearly marking the frozen-window exception. The key is excluded from later snapshots.

A tranche follows the finite state machine `FREE`→`RESERVED(W)`→`LIABLE(W)`→`RELEASED` or `RESERVED/LIABLE`→`SLASHED`. Candidate acceptance marks every used window liable. Release requires the window to be below `minAdmissibleSlot`, its normal/recovery and L1 reorg horizons to have passed, and the evidence delay to have expired. A bitmap per snapshot and a ring of at most `MAX_LIVE_WINDOWS` replace unbounded slash-history arrays.

4.2 Authenticated snapshot

For aligned window W , let its start timestamp be `GENESIS_TIMESTAMP+384W`. The target is the greatest `BEACON_GENESIS_TIME+12k` not later than eight L1 epochs before that start; L2-genesis alignment is irrelevant. Define `sealDeadline(W)=windowStart-H_LOOK`. A seal is valid only when its source has `F_FINAL` depth and `block.timestamp<sealDeadline(W)`. At or after the deadline an unsealed window is permanently all-`VACANT`; a late transaction cannot change it. The contract performs this procedure:

1. For $j = 1, \dots, 64$, query EIP-4788 at $q=\text{targetTimestamp}+12j$; missing timestamps revert and are skipped. The first successful query identifies the *carrier* execution block. EIP-4788 returns its parent beacon-block root, not the carrier root.
2. Supply the carrier execution header. EIP-2935 authenticates its hash; the header timestamp equals q and its `parent_beacon_block_root` equals the EIP-4788 result. Fork-specific SSZ branches in that parent authenticate its slot and one execution payload’s `blockHash`, `stateRoot` and `prevRandao`. The parent slot must be at or before target, its payload timestamp must equal its beacon-slot time, and its payload block hash must equal the carrier header’s parent hash. No carrier, a later parent, or a fork/gindex mismatch means all `VACANT`.
3. Verify Merkle–Patricia storage proofs from that execution state root for the registry header

and `registryRoot`. The caller supplies all 64 canonical serialized cells, including empty cells; recompute their six-level Keccak tree and require equality to `registryRoot`. Filter eligibility and order the entries, then store the entry-root and seed. The fixed leaf count proves completeness with one MPT path rather than 64 independent storage paths. The source payload and carrier have both reached `F_FINAL` depth.

`sealWindow` is permissionless. An optional maintenance distributor may pay its event in a later transaction. A missing seal fails closed to `VACANT`; it cannot give an address an unauthorized ticket. The snapshot age plus 64-slot carrier scan, finality and seal margin fits the eight-epoch geometry and remains far below EIP-4788's 8,191-slot history window. Consensus constants include the Deneb SSZ generalized indices; an L1 fork that changes them requires a protocol-version upgrade, never runtime guessing.

For Deneb, Electra and Fulu, the generalized index from `BeaconBlock` root to slot is 8 and to `body.execution_payload` is 201. Within the composed tree, payload `state_root`, `prev_randao`, `timestamp` and `block_hash` are 6,434, 6,437, 6,441 and 6,444 respectively. The seal input names the L1 fork digest; only these configured digests are accepted. Gloas or any fork that changes the container requires new constants, fixtures and a protocol-version upgrade before its activation.

4.3 Consensus byte encoding and allocation

All hashes use Ethereum legacy Keccak-256. Concatenation below is packed fixed width; strings are the literal ASCII bytes without a length prefix:

$$\begin{aligned} seed_W &= H(\text{"slot-chain-seed-v1"} \parallel u256(chainId) \\ &\quad \parallel u256(protocolVersion) \parallel u256(W) \parallel bytes32(prevRandao)), \\ qTie_i &= H(\text{"slot-chain-quota-tie-v1"} \parallel seed_W \parallel u64(regIndex_i)), \\ sTie_{p,i} &= H(\text{"slot-chain-slot-order-v1"} \parallel seed_W \parallel u16(p) \parallel address20(i)). \end{aligned}$$

Starting from zero, capped D'Hondt awards each of 384 tickets to the unsaturated entry maximizing $bond/(allocation+1)$, with $Q_MAX=76$. Quotients are compared by cross multiplication; a `uint192` bond times Q_MAX+1 cannot overflow `uint256`. Equal quotients use smallest `qTie`. Remaining tickets are `VACANT`; six addresses are required to fill a window.

Placement processes positions in order. A real address is feasible if it has a ticket remaining, differs from the preceding real address, and removing it preserves `remaining[a] ≤ otherRemaining+1` for every real address. Choose the feasible value with smallest `sTie`; `VACANT` may repeat. The executable model and its golden vectors are normative test fixtures.

5 Blocks, promises and proof statement

5.1 Signed header

A builder signs exactly:

```
(chainId, protocolVersion, verifyingContract, slot, parentHash,
blockHash, stateRoot, bodyRoot, anchorNumber, anchorHash,
forceRoot, forceCutoff, messageStart, messageEnd, dataManifestRoot,
```

```
coinbase, tier, admissionVersion, episode, recoveryRevision, recoveryId)
```

The normative EIP-712 type string is:

```
SlotChainBlock(
  uint256 chainId,uint256 protocolVersion,address verifyingContract,
  uint64 slot,bytes32 parentHash,bytes32 blockHash,bytes32 stateRoot,
  bytes32 bodyRoot,uint64 anchorNumber,bytes32 anchorHash,bytes32 forceRoot,
  uint64 forceCutoff,uint64 messageStart,uint64 messageEnd,
  bytes32 dataManifestRoot,address coinbase,uint8 tier,
  uint64 admissionVersion,uint64 episode,uint64 recoveryRevision,
  bytes32 recoveryId)
```

The hashed ASCII has one space between each field type and field name and no other whitespace; displayed line breaks, indentation and spaces after commas are typography only. The exact single-line literal and a standalone Keccak implementation are in `commitment-model.py`. TYPEHASH is legacy Keccak-256 of those ASCII bytes and equals the concatenation of these two fragments:

```
0x0a0bfc3b8ba52a166f662055d0989146
5d916aa486808e78105890e98d65777f
```

The EIP-712 domain is `("SlotChain", "2", chainId, verifyingContract)`. Signatures use low-s secp256k1 and reject zero/recovered-zero addresses. The execution `blockHash` excludes the off-chain builder signature, avoiding a circular hash definition.

5.2 Three validity tiers

1. **Normal signed.** Every block is signed by its scheduled builder under the normal window's frozen `admissionVersion`. Slots strictly increase, each slot is no later than `currentL2Slot+CLOCK_SKEW`, and the first slot strictly exceeds the canonical base slot. Parent gaps are at most `G_MAX=64`.
2. **Recovery signed.** The first block extends the L1-final canonical tuple, binds the active episode and current recovery revision, and may cross an unbounded time gap. Every block still requires its scheduled builder signature under the recovery round's current admission version.
3. **Escape unsigned.** Exactly one deterministic block extends the L1-final canonical tuple during recovery. It has no builder signature, no beneficiary-controlled coinbase, no discretionary transaction and no blob body. It contains only the protocol anchor and the deterministic maximum prefix of the activation-frozen forced queue. When the queue is empty an SLA-triggered episode may commit an empty anchor-only escape block. Its slot is `max(activationSlot+ESCAPE_OFFSET, canonical.tipSlot+1)` and its anchor, queue root/cutoff, block hash and state root are unique functions of the current recovery round and proved execution result.

Every candidate uses exactly one `batchAnchor`; every block repeats that anchor and its authenticated `forceRoot/forceCutoff`. The supplied RLP execution header hashes to the EIP-2935 value, its timestamp is no later than the first L2 block timestamp, and MPT proofs under its state root authenticate the forced queue and any other historical contract state. Requiring one

anchor, not up to 4,096 independent anchors, bounds both circuit and L1 work. Cutoffs therefore do not change within a candidate; the first blocks drain the frozen FIFO prefix and later blocks see an empty prefix.

The circuit verifies parent linkage, strict slot monotonicity, schedules, signatures for tiers 1–2, version binding, anchor/header/MPT authentication, forced-prefix ordering, execution, exact body/data coverage and public outputs. It rejects more than 4,096 blocks, 12 schedule windows, 16 sealed sessions or 2,100 referenced blob records.

5.3 Verifier statement ABI

The L1 verifier accepts `verify(proof,uint256[2]digestLimbs)`. Settlement constructs the following Solidity struct in exactly this order and computes `statementHash` as legacy Keccak of the ASCII domain "slot-chain-statement-v1" followed by `abi.encode(S)`:

```
S = (uint256 chainId, uint256 protocolVersion,
    address verifyingContract,
    uint8 tier, bytes32 baseTupleHash, bytes32 endTupleHash,
    bytes32 candidateCommitment, uint64 blockCount,
    uint64 firstSlot, uint64 tipSlot,
    uint64 anchorNumber, bytes32 anchorHash,
    bytes32 anchorStateRoot, uint64 anchorTimestamp,
    bytes32 forceRoot, uint64 forceCutoff,
    uint64 startCursor, uint64 endCursor,
    uint64 admissionVersion,
    bytes32 admissionRoot,
    uint64 episode, uint64 recoveryRevision, bytes32 recoveryId,
    bytes32 scheduleRootsCommitment, uint8 scheduleWindowCount,
    bytes32 sessionRefsCommitment, uint8 sessionCount,
    uint16 dataRecordCount, bytes32 executionOutputsCommitment,
    address proofBeneficiary)
```

`digestLimbs[0]` and `[1]` are respectively the high and low 128 bits of `statementHash`; neither is reduced modulo the proof field. The circuit recombines them and proves the exact preimage. Settlement independently checks the base tuple, current/frozen versions and recovery round, recomputes the bounded schedule-root and sealed-session-reference commitments from calldata, and writes exactly `endTupleHash`. `proofBeneficiary` is a proof input, never builder-signed block data or coinbase. Copying proof bytes cannot redirect a later optional reward.

5.4 Preconfirmation semantics

A builder signature proves authorship and makes same-slot equivocation slashable. It does not prove that the body reached a prover or that the next builder saw it. A later scheduled builder may sign a profitable skip that the protocol cannot distinguish from non-receipt. Accordingly:

- before L1 candidate acceptance, a promise is *observed*;
- after acceptance and before close, it is *provisionally proved*;
- after normal close or recovery commit, it is *canonical*; and

- only the underlying L1 finality policy makes it *L1-final*.

Wallets and applications MUST NOT display the first two states as final. An escape commit explicitly revokes every omitted observed or provisionally proved promise and returns omitted transactions to the mempool.

6 Blob data authentication

Data is posted into publisher-owned, bonded sessions. A session ID is

```
H("slot-chain-session-v1" || u256(chainId) || address20(contract) ||
  address20(owner) || u64(ownerNonce))
```

At most two sessions per owner and 1,024 globally may be live; each contains at most 2,100 records. Opening uses the first free ring cell, after deterministically evicting expired cells in index order. A full live ring rejects discretionary publication but cannot block the forced/escape lane. Rent covers worst-case garbage collection.

Only the owner may append, and a candidate may reference only a session made immutable by `sealSession`. Sealing fixes `(root, count, expiry)`; there is no `unseal` or `append-after-seal` operation. A candidate supplies at most 16 sorted unique `(sessionId, count, root)` references. At submission each must equal a live sealed session. Normal acceptance requires `expiry ≥ normalDeadline + REORG_MARGIN_SECONDS`; recovery requires `expiry ≥ block.timestamp + REORG_MARGIN_SECONDS`. Opening requires its expiry to cover proving, settlement and the reorg margin:

$$\begin{aligned} \text{expiry} &\geq \text{now} + P_PROVE_MAX + W_SETTLE_SECONDS \\ &\quad + REORG_MARGIN_SECONDS, \\ \text{expiry} &\leq \text{now} + DATA_TTL. \end{aligned}$$

For every blob attached to `postData(session, manifests[])`, the contract:

1. obtains `versionedHash = BLOBHASH(i)` from the same transaction;
2. computes legacy Keccak over the packed fixed-width sequence `"slot-chain-data-fs-v1", u256(chainId), u256(protocolVersion), bytes32(sessionId), bytes32(versionedHash), bytes32(bodyRoot), address20(publisher), u64(validUntil)`, interprets it big-endian and reduces it modulo the BLS scalar-field modulus to obtain `z`;
3. calls the EIP-4844 point-evaluation precompile with exactly `versionedHash32 || z32 || y32 || commitment48 || proof48`; and
4. appends the exact leaf from §A to the session MMR.

The proof receives blob polynomials privately, recomputes $P(z) = y$, decodes the canonical blob codec and recomputes `bodyRoot`. A body byte string is `u32(txCount) || (u32(txLength) || signedRawTx)*`; its root is `H("slot-chain-body-v1" || u32(byteLength) || bodyBytes)`. Blob payload is `u32(byteLength) || bodyBytes || zeroPadding`, split into 31-byte chunks; each blob field element is `0x00 || chunk31`. Exactly 4,096 elements are used, unused payload bytes are zero, and noncanonical encodings reject. A manifest uses increasing unique indices and partitions every discretionary body byte exactly once. Duplicate, extra or uncovered data is invalid.

Data publication has no consensus-coupled reimbursement. A separate service may pay it, but empty payment state cannot revert canonical settlement.

7 Settlement and recovery state machine

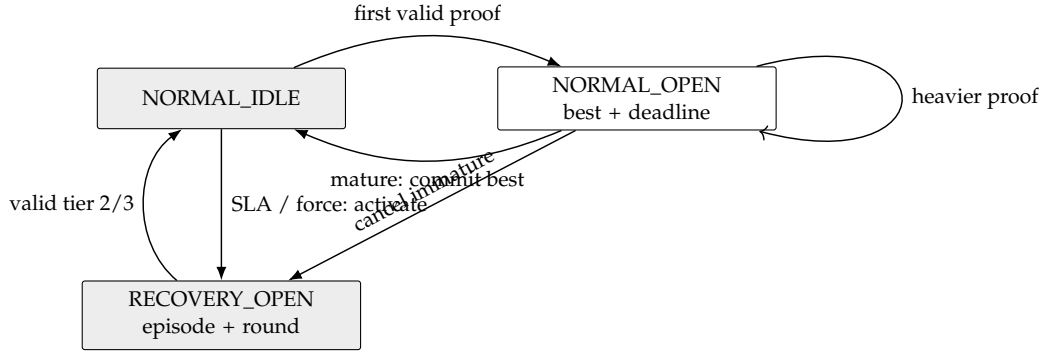


Figure 1: The complete mode machine. Payments are outside every arrow.

7.1 Normal candidates

The first valid tier-1 candidate extending the canonical tuple opens a normal window. It freezes `baseCanonical`, `deadline=block.timestamp+W_SETTLE_SECONDS`, and `minAdmissibleSlot=max(0, currentL2Slot-DELTA_TIP)`, and current `admissionVersion`. Before accepting the first candidate, Settlement computes the prospective deadline; every candidate must consume all FIFO messages whose `dueAt<=deadline`. Later candidates use the same base, deadline, minimum and admission version. Their total order is lexicographic:

(block count descending, tip slot descending, tip hash ascending).

Only a strictly better candidate replaces `normalBest`; provisional replacement changes no canonical state and pays nobody. At or after the deadline, `sync()` commits the stored best only if its end cursor still covers the required prefix. Because `FORCE_DELAY>=W_SETTLE_SECONDS+T_INCLUDE_MAX`, a message appended after the window opens cannot newly become due before that deadline. Settlement then recomputes recovery causes against the new tuple.

7.2 Activation

While normal, `sync()` opens one persistent recovery episode if either:

- `currentL2Slot>canonical.tipSlot+DELTA_FINAL_LAG`; or
- the forced queue head satisfies `enqueuedAt+FORCE_DELAY<=block.timestamp`.

A due forced head has precedence over an immature normal window, which is canceled. At a mature deadline, a best that consumes the required due prefix commits before causes are recomputed; an omitting best is canceled. No best is promoted across activation. The contract increments episode, records causes and the base-tuple hash, performs the SLA seat termination/burn at most once, then opens recovery round one.

Every round freezes `roundStartSlot`, the preceding L1 block number/hash, its times-

tamp, current forceRoot/forceCutoff, current admission version, and escapeSlot=max(roundStartSlot+ESCAPE_OFFSET,canonical.tipSlot+1). Its expiresAt is GENESIS_TIMESTAMP+escapeSlot+DELTA_TIP. The exact ID is:

```
recoveryId = H("slot-chain-recovery-v2", u256(chainId), address20(contract),
               u64(episode), u64(revision), bytes32(baseTupleHash),
               u64(roundStartSlot), u64(anchorNumber), bytes32(anchorHash),
               bytes32(forceRoot), u64(forceCutoff), u64(admissionVersion),
               bytes32(admissionRoot), u64(escapeSlot), u8(causes))
```

At `block.timestamp>expiresAt`, and never before, `sync()` replaces the expired target with revision $r + 1$ and fresh round fields, then returns SYNCED. It does not change episode/base/cause, re-burn, terminate, promote or pay. The old proof is already inadmissible, so permissionless refresh cannot grief a still-live proof. New queue appends enter the refreshed cutoff. An envelope admitted during recovery receives `dueAt=max(enqueuedAt+FORCE_DELAY,currentRound.expiresAt+1)`; therefore it cannot become due while a frozen round that omits it remains admissible.

7.3 Recovery acceptance

Recovery is first-valid, not heaviest. A tier-2 or tier-3 candidate MUST:

1. extend the exact current stored canonical tuple;
2. bind current episode, revision, recoveryId, frozen admission version, anchor and queue target;
3. land only after `block.number-anchorNumber>=F_L1`, have first slot at least `roundStartSlot`, tip no older than `DELTA_TIP`, and no slot later than wall clock plus skew;
4. consume the exact maximum forced-message prefix from the current cursor, bounded by the activation-frozen cutoff; and
5. satisfy its tier-specific signature/data restrictions and validity proof.

The first valid L1 transaction atomically commits the tuple, records the current L1 block *number only*, clears the recovery episode and returns to normal. Later proofs are stale. Candidate rejection cannot undo activation because of the early-return wrapper in §3.

8 Forced queue and unsigned escape

A forced envelope is `(kind,sender,nonce,chainId,rawTx,byteLength,accountedGas,maxFee,validUntil,refundAddress,enqueuedAt,dueAt,deposit)`. For `kind=0`, admission canonically decodes the EIP-2718 signed raw transaction and requires its recovered sender, nonce, chain ID, gas limit and fee cap to equal the outer fields. For `kind=1`, only the configured L1 BridgeAdapter may enqueue a canonical deposit/system call in the same transaction that accepts the L1 bridge message; the L2 inbox transition is idempotent, has no external call and is proved never to revert. Other kinds reject. Both paths compute intrinsic/accounted gas and require `accountedGas=max(tx.gasLimit,intrinsicGas,MIN_FORCE_ACCOUNTED_GAS)`, `0<accountedGas<=5,000,000`, `byteLength=rawTx.length<=131,072`, a nonexpired `validUntil`, and a deposit covering execution, proof credit and permanent storage. Bytes are published in the enqueue transaction's L1 calldata and their Keccak is in the queue leaf. Blobs

are forbidden for this availability path; reconstruction assumes access to canonical archival L1 calldata, the same assumption used by bridge indexers.

Each proved block consumes the unique maximum FIFO prefix satisfying all three bounds simultaneously:

$$count \leq 64, \quad \sum byteLength \leq 1,048,576, \quad \sum accountedGas \leq 20,000,000.$$

Every admitted item individually fits all three. The circuit exposes (start,end,count,totalBytes,totalAccountedGas,forceRoot,forceCutoff) and proves no skip or early stop. Launch enforces $ANCHOR_GAS_MAX + FORCE_GAS_BUDGET + SYSTEM_GAS_MARGIN \leq L2_BLOCK_GAS_LIMIT$ and the analogous witness-byte bound.

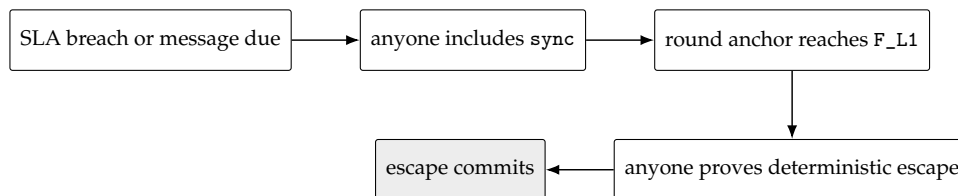
Every L2 body orders the protocol anchor transaction first, that block's forced prefix contiguously second, and discretionary transactions last. At execution, an expired envelope, bad current nonce, insufficient upfront balance or currently inadequate fee cap is consumed with a deterministic discard code and has no Ethereum nonce/receipt effect. An upfront-valid kind-0 transaction executes as ordinary Ethereum: success or revert both increment nonce, charge gas and emit a receipt. A revert is *included*, never mislabeled discarded. Kind-1 executes its total system transition and cannot use discard semantics. Unused deposit is credited to refundAddress by a non-consensus module; failure there cannot change queue consumption.

For a signed block, forceRoot/forceCutoff are the queue state authenticated by that block's L1 anchor, and the circuit consumes the maximum prefix only up to that cutoff. A later L1 append therefore cannot invalidate an already signed block or in-flight proof. All blocks in one candidate use the same cutoff. A due current head cannot be bypassed: sync() cancels an immature window, or closes a mature best only if it consumed the deadline-required prefix, before opening recovery.

The 1,500-second force delay applies to the queue head, not to an arbitrary position behind existing prepaid work. An admitted sender can compute its worst-case number of escape blocks from the gas ahead at admission. As with L1 itself, the protocol does not promise backlog-independent latency against an adversary willing to buy all available capacity; it promises that builders cannot selectively veto the FIFO head.

An escape block is deterministic. Its timestamp is its frozen slot, coinbase is the protocol sink, base fee follows the ordinary formula, randomness is derived from the frozen authenticated anchor, and its only transactions are anchor plus the frozen forced prefix. The circuit proves those facts. No party gains timestamp, ordering or MEV discretion by producing the proof.

For a cartel with no usable builder block, the recovery sequence is:



With the initial bounds, activation inclusion (120 s), the frozen ESCAPE_OFFSET=1,900s (covering T_DEPTH_MAX=900s, proving at 900 s and 100 s margin), submission inclusion (120 s), and clock margin (24 s) total 2,164 seconds, below DELTA_FINAL_LAG=3,600s. Tip admissibility is separate: the escape slot is at the end of the offset, so only submission and skew—144 seconds—must fit DELTA_TIP=1,200s when proof generation meets its bound.

9 Economics, slashing and payments

Event	Objective evidence	Effect
Same-slot equivocation	Two valid distinct signed headers for one key/slot	Slash that schedule window's tranche once; tombstone key; emit reporter entitlement.
Aggregator SLA breach	Final lag exceeds threshold after mature close	Terminate and burn penalty bond once; open recovery; promote standby if any.
Force-only activation	Due queue head	No aggregator penalty; open recovery.
Skip or absence	None that distinguishes fault from non-receipt	Not slashable; promise may be revoked.
Invalid proof/data	Verifier failure	Reject; no canonical or payment effect.

Equivocation evidence is idempotent per (builder, window); separate windows slash separate tranches. L_LEASE is calibrated against at most 76 assigned slots but is not insurance for unbounded signed value. Tombstoning is atomic with the first valid slash; future snapshots exclude the key.

Canonical settlement performs no reward calculation, balance write or external call. Its event records candidateId and the proof-bound beneficiary. In a separate transaction an optional RewardDistributor checks that ID against Settlement and materializes at most the funded cap from forced deposits, the terminated seat's recovery escrow and an optional reserve. Failure or exhaustion affects only that second transaction. Burns are never recycled into the same episode's bounty. Normal service/data compensation likewise cannot affect candidate validity or canonical commit.

10 Security and liveness analysis

Adversary/failure	Protocol result	Bound or residual
Invalid execution or forged state	Proof rejects.	Finalized-state safety reduces to proof-system and L1 safety.
Aggregator of-fline/byzantine	SLA activation terminates it; any prover uses tier 2 or 3.	At most final-lag threshold plus 2,164 s under stated inclusion/proof bounds.
No seat or no standby	State remains valid; escape has no seat dependency.	Same protocol bound; reward may be zero.
All builders collide/absent	Unsigned escape advances anchor and forced queue.	Meaningful discretionary throughput falls to forced envelopes; no builder censorship veto.
Builder equivocation	Competing signed branches; one may win normal order.	Direct per-window slash; pre-close promises remain revocable.

Builder skips/withholds body	Honest tail may be unprovable or omitted.	Not objectively slashable; eventual escape restores state progress but may revoke promises.
Data-session spam	Sessions are publisher-owned and bonded.	Attacker fills only its session; escape uses no blob session.
Transparent-mempool front-run	First valid recovery transaction wins.	Proof statement binds beneficiary, episode, revision and outputs; copying cannot redirect payment or state.
Payment-vault ex-haustion	Claim is zero or partial.	Never blocks safety or canonical liveness; economic liveness is conditional.
L1 reorg within policy	Contract state, burns and claims roll back together.	Clients replay canonical logs and proofs against new version/hash.
L1 censorship/outage	Neither activation nor proof can land.	Outside model after T_INCLUDE_MAX; no L2 protocol can force its L1 contract to execute.

The escape lane establishes strong transaction-entry permissionlessness even when the capital-ranked builder set is captured. It does not make the normal builder market egalitarian: top-64 ranking and per-window capital remain a meaningful entry barrier. The product must distinguish these two claims.

11 Implementation blueprint

11.1 L1 modules and bounded storage

- **BuilderRegistry**: entries, tranche bitmap, tombstone and bounded liability ring.
- **ScheduleOracle**: sealed window root/seed in a ring covering MAX_LIVE_WINDOWS=262. In window units this is at least the ceiling of lookahead, candidate, settlement, recovery, evidence and reorg horizons:

$$\begin{aligned} &\text{ceil}(\text{H_LOOK}/384) + \text{MAX_CANDIDATE_WINDOWS} + \\ &\text{ceil}((\text{W_SETTLE_SECONDS} + \text{DELTA_FINAL_LAG} + \text{DELTA_TIP} + \\ &\quad \text{REORG_MARGIN_SECONDS} + \text{EVIDENCE_DELAY_SECONDS}) / 384) + 2 \end{aligned}$$

An entry is overwritten only if its release predicate is true and no stored normal best references it.

- **DataSession**: a 1,024-cell global ring, at most two live cells per owner, each with owner/nonce ID, 2,100-leaf MMR frontier, sealed root/count and expiry; no on-chain leaf array. Expired eviction is deterministic.
- **ForcedQueue**: append-only envelope commitment, cursor and deposits; payload bytes are permanent L1 calldata and never blobs.
- **Settlement**: PREACTIVE flag, canonical tuple, one normal best, recovery episode/round and seat pointer. Rewards are separate. No loop is proportional to chain age.

All setters enforce parameter inequalities atomically and are disabled after the launch freeze except through the existing delayed governance path. Every counter has an explicit width and checked increment; no sentinel shares a legal value.

11.2 Historical authentication

EIP-2935 authenticates normal candidate and schedule carrier headers. A recovery round captures its immediately preceding block hash with `blockhash(block.number-1)` and later compares candidates with that stored value; it never queries an expired history entry. EIP-4788 plus SSZ branches authenticates the schedule's parent beacon/execution payload; its state root plus MPT proofs authenticates the registry. Deployments lacking either primitive require an equivalently verified oracle and separate review.

11.3 Reorg rule

The canonical L1 log is the journal. A reorg truncates and replays, in order: canonical tuple, mode/version, normal best/deadline/frozen admission, episode/round, seat termination/burn, tombstone/tranche state, forced cursor, data-session root/count, and reward eligibility. Off-chain workers index by `(chainId, contract, blockHash, logIndex)` and discard orphaned jobs. Withdrawal/bridge execution waits for the configured L1 finality policy.

11.4 Genesis and migration

Deployment occurs at least `D_SNAP+F_FINAL+sealMargin` before activation, so first live sources are post-deployment and sealable. New Settlement begins PREACTIVE; registry, scheduling and data preparation may run, but every candidate rejects.

Forced user transactions and kind-1 bridge messages live in one shared `ForcedQueue` used by old and new settlement contracts, so there is no queue-root/cursor copy race. Routing the legacy bridge through the audited `BridgeAdapter` is a migration prerequisite; deployment is blocked if any old pending message is not represented exactly once in this queue. At or after the announced activation time, one governance executor transaction calls `old.freezeAndExport(new)`. The old contract first performs its objective sync, cancels any remaining provisional window, permanently disables candidate ingress and exports its exact current canonical tuple, forced cursor and bridge withdrawal cursor to the new contract. The new contract checks the shared queue root/cursor, imports those values and changes PREACTIVE to NORMAL before the same transaction returns. Any revert rolls both contracts back. Bridge claims are gated by the exported withdrawal cursor/root. Thus no L1 ordering interval can advance the old state between snapshot and import, and no slot or message is accepted by both or neither verifier.

12 Parameters and enforced geometry

Parameter	Initial value	Enforced relation
L2 slot / schedule window	1 s / 384 slots	Aligned by genesis timestamp.
N_MAX, Q_MAX	64 / 76	Six addresses fill 384 tickets.
BOND_MAX	$2^{192} - 1$	$\text{bond} * (\text{Q_MAX} + 1) < 2^{256}$.

D_SNAP, F_FINAL	8 / 2 L1 epochs	Strictly exceeds scan + finality + seal margin + lookahead.
H_LOOK	768 L2 slots	Guaranteed by snapshot geometry with one-window slack.
G_MAX	64 L2 slots	Tier-1 parent gap only.
W_SETTLE_SECONDS	1,200 s	Timestamp deadline, never L1 block count.
DELTA_TIP	1,200 s	At least submit inclusion + skew = 144 s after the frozen escape slot.
DELTA_FINAL_LAG	3,600 s	At least activation + escape offset + submit + skew = 2,164 s.
F_L1; T_DEPTH_MAX	64 L1 blocks; 900 s	Stored recovery-anchor depth and assumed maximum time to reach it.
ESCAPE_OFFSET	1,900 s	At least depth-time bound + proof bound + margin.
FORCE_DELAY	1,500 s	At least settlement window plus 120-s inclusion bound.
Forced prefix	64 items; 1,048,576 B; 20m gas	Each item at most 131,072 B and 5m accounted gas.
Candidate caps	4,096 blocks; 12 windows; 1 anchor; 16 sessions; 2,100 records	Independent verifier/circuit limits.
DATA_TTL	86,400 s	Greater than candidate, settlement, proof and reorg path.
REORG_MARGIN, EVIDENCE_DELAY	1,800 / 86,400 s	Data resubmission and finite liability retention.
Live windows / sessions	262 / 1,024	Fixed rings with safe-eviction predicates.
EIP history	8,191 entries	Greater than every normal-anchor and schedule path.

Launch tooling evaluates every inequality in both seconds and its native clock unit. Governance cannot set a value that violates a relation.

13 Production gates and verdict

The consensus design is fixed enough to implement. Production deployment is blocked until every gate below produces a versioned artifact and passes:

1. **Proof performance.** Independent tier-1, tier-2 and worst-case tier-3 proofs, including 4,096 blocks and 2,100 data records, finish within 900 seconds on the published minimum hardware. Peak RAM, recursion setup and failure recovery are measured; a miss requires changing parameters and another design review.
2. **Contract gas.** sealWindow, postData, normal submit, sync, recovery submit, slash and garbage collection fit current L1 gas limits with 30% margin. Fuzzing includes empty registry, zero seat, full data session, full queue prefix and maximum history age.

3. **Cryptographic conformance.** Solidity, Go, Rust and circuit code match the Keccak/EIP-712 golden vectors, SSZ generalized indices, MPT proofs, KZG Fiat–Shamir transcript and exact body manifest. Two independent implementations generate every vector.
4. **State-machine verification.** Stateful fuzzing models EVM rollback, same-block ordering, reorg replay, stale versions, activation/close coincidence, message consume-and-discard and storage reclamation. The executable models in Appendix D remain green.
5. **Economics.** Auction bond, per-window tranche, forced-envelope deposit, storage bond and proof credits are calibrated under blob/L1 fee spikes. The published liveness claim explicitly becomes conditional above fee caps.
6. **Operations.** At least two independent prover stacks, public data session mirrors, schedule sealers and recovery bots complete a shadow-fork outage exercise. The exercise removes every builder and seat, forces a user transaction, reorgs the first recovery attempt, and still reaches finality.
7. **External review.** Separate proof-system, Solidity, bridge and economic audits close all critical/high findings; the migration rehearsal and emergency governance path are in scope.

***Verdict.** The earlier fallback architecture was not implementation-ready: it depended on a scheduled builder, could revert activation inside a rejected submission, made payment a commit precondition, and could not bind whole-window data with a six-blob cap. This revision removes those dependencies and fixes a single consensus path. It is implementation-ready as a specification. It is not production-ready as a deployed system until the seven empirical gates pass; claiming otherwise from a document and Python assertions alone would be unsound.*

A Normative commitment encodings

All integers in packed commitments are unsigned big-endian at the named width; addresses are 20 bytes, hashes are 32 bytes, and ASCII domains have no length prefix or trailing NUL. `H` is Ethereum legacy Keccak-256. No `abi.encodePacked` call may contain a dynamic field. Raw transaction bytes are represented by `H(rawTx)` plus an explicit `u32(byteLength)`.

The canonical tuple, candidate, schedule list, sealed-session list and execution outputs use these exact encodings:

```
H("slot-chain-canonical-v1" || tipHash || u64(tipSlot) || stateRoot ||
  u64(messageCursor) || u64(canonicalizationBlock) || winningDataCommitment)
H("slot-chain-candidate-v1" || baseTupleHash || u16(n) ||
  (u64(slot) || blockHash || bodyRoot || u64(messageEnd))* )
H("slot-chain-schedule-list-v1" || u8(count) ||
  (u64(window) || entryRoot || seed)* )
H("slot-chain-session-list-v1" || u8(count) ||
  (sessionId || u16(recordCount) || root)* )
H("slot-chain-outputs-v1" || stateRoot || transactionsRoot || receiptsRoot ||
  logsBloomHash || withdrawalsRoot)
```

Here $n \leq 4096$; schedule windows are ascending and sessions are ascending by ID. These hashes are respectively the statement's `baseTupleHash`, `candidateCommitment`, `scheduleRootsCommitment`, `sessionRefsCommitment` and `executionOutputsCommitment`; unsorted or duplicate lists reject.

The 64-cell registry leaf at index i is

```
H("slot-chain-registry-leaf-v1" || u8(i) || u8(present) ||
  address20 || u192(bond) || u64(registrationIndex) ||
  u64(effectiveSlot) || bytes32(trancheRoot) || u64(tombstoneSlot))
```

An empty cell has `present=0` and every following byte zero. A present cell has `present=1`; `UINT64_MAX` means not tombstoned. At tree height $h = 0, \dots, 5$, the parent is:

```
H("slot-chain-registry-node-v1" || u8(h) || left || right)
```

There is no padding because there are exactly 64 leaves.

Each registry cell's tranche ring is a 512-leaf tree. Leaf index j is:

```
H("slot-chain-tranche-leaf-v1" || u16(j) || u64(window) || u8(state) ||
  u192(amount) || u64(liableUntil))
```

An empty leaf has `window=UINT64_MAX`, state zero and remaining fields zero. Its nine node levels use `H("slot-chain-tranche-node-v1" || u8(height) || left || right)`. The snapshot supplies one inclusion proof for the requested window per eligible cell.

Forced leaf i is legacy Keccak over this exact packed sequence:

```
"slot-chain-force-leaf-v1" || u64(i) || u8(kind) ||
address20(sender) || u64(nonce) || u256(chainId) || bytes32(rawTxHash) ||
```

```
u32(byteLength) || u64(accountedGas) ||
u256(maxFee) || u64(validUntil) || address20(refundAddress) ||
u64(enqueuedAt) || u64(dueAt) || u256(deposit)
```

The empty queue root is $H(\text{"slot-chain-force-init-v1"})$. Append leaf L_i as $\text{root}[i+1]=H(\text{"slot-chain-force-node-v1"}||\text{root}[i]||L_i||u64(i+1))$. The count is therefore bound at every step. A queue proof supplies the prior root and contiguous leaves; there is no membership/nonmembership format that can skip an index.

A data leaf is:

```
H("slot-chain-data-leaf-v1" || bytes32(sessionId) || u16(index) ||
  bytes32(versionedHash) || bytes32(bodyRoot) || address20(publisher) ||
  u64(validUntil) || u256(z) || u256(y))
```

MMR append merges equal-height peaks as $H(\text{"slot-chain-data-node-v1"}||u8(\text{height})||\text{left}||\text{right})$. Peaks are stored left-to-right in strictly decreasing height. The root bags them rightmost-to-leftmost as $H(\text{"slot-chain-data-bag-v1"}||u16(\text{count})||u8(\text{peakCount})||(u8(\text{height})||\text{peak})*)$. An inclusion proof provides the leaf index, merge siblings bottom-up with a left/right bit at each height, then every other peak in this bag order; duplicate heights, a count inconsistent with peak heights, or unused proof elements reject. The empty MMR is the bag hash with zero count and zero peaks.

The recovery ID is the encoding in §7. Schedule leaf/root, seed and tie hashes are as in §4; body and blob encodings are as in §6. `commitment-model.py` is the normative executable fixture. Its initial vectors are:

TYPEHASH	0a0bfc3b8ba52a166f662055d09891465d916aa486808e78105890e98d65777f
registryRoot	0bf297d7b9b6a5529a319a06cb08484923a89bab15d51f8baeaf5c30bebdf3fd
forcedRoot	0e5416c12bf86837bad8fd0370f247683a401ceaa80f45e4a3fca52876ad448d
MMR(root,2)	f06fd107666ba9e51ad09606ba9c9f1f1c16177f586167f87b16f333806d485d
recoveryId	9970a429a00e56b8abc5e7fcb4a4a35795507e949821be66f2f737e888ded285
bodyRoot	0f4e161a46c8b18c2a86f23a0a4e7169a838a12af8b389f65e97b547a99707e9

Release CI must reproduce every vector in Solidity, Go, Rust and the circuit.

B Normative transition ordering

Every external candidate entry point implements the following structure:

```
function submit(input) returns (Status) {
    if (mode == PREACTIVE) return REJECTED_PREACTIVE;
    if (sync()) return SYNCED;      // successful transaction, never revert
    if (mode == NORMAL) return submitNormal(input);
    return submitRecovery(input);
}
```

```
function sync() returns (bool changed) {
    if (mode == RECOVERY && now > round.expiresAt) {
        rollRound();                // same episode/base/burn; new target
    }
}
```

```

        return true;
    }
    if (mode == NORMAL && forcedHeadDue && normal window immature) {
        cancel normal window;
    } else if (normal deadline matured) {
        if (best covers requiredCursor(deadline)) commit(normalBest);
        else cancel normal window;
    }
    if (mode == NORMAL && (finalLagBreach || forcedHeadDue)) {
        openRecoveryEpisodeAndRound();
        if (finalLagBreach) terminate incumbent once;
        return true;
    }
    return whetherCloseOrCancelChangedState;
}

```

No transfer, reward calculation, unbounded loop or caller-controlled external call occurs in canonical settlement. Optional reward materialization is a later transaction in a separate module.

C Rejected alternatives

1. **Heaviest fallback.** Rejected because an observer cannot prove it has seen the globally heaviest off-chain tail; transparent races can prevent a stable target. Recovery is episode-bound first-valid.
2. **Scheduled-builder-only fallback.** Rejected because the same cartel that stalls normal production also controls recovery. Tier 3 is deterministic and unsigned.
3. **Promote the pre-activation normal best.** Rejected because it was accepted under different cutoff/version semantics. Only a mature best that covers the required forced prefix closes; immature state is canceled.
4. **Atomic reward-funded commit.** Rejected because an empty seat or vault becomes a consensus deadlock. Reward materialization is a separate non-authoritative transaction.
5. **One same-transaction blob set.** Rejected because Ethereum exposes at most a handful of blobs per transaction while a candidate spans thousands of blocks. Publisher-owned authenticated sessions separate posting from proof.
6. **One KZG opening at a prover-chosen point.** Rejected because it does not soundly tie declared bodies to the blob. The challenge commits to both blob hash and body root, and the circuit recomputes the evaluation and body root.
7. **Arithmetic snapshot slot without EIP-4788 parent semantics.** Rejected. A bounded forward carrier search authenticates the parent source or fails closed.
8. **Bond as insurance.** Rejected without an on-chain reservation ledger. The tranche is deterrence only.

D Executable consensus models

lookahead-model.py reports 29 assertions over legacy Keccak, EIP-4788 carrier/parent se-

lection, immutable seal deadlines, post-seal tombstones, eligibility-before-ranking, capped D'Hondt and feasible placement, including fixed seed and complete-schedule vectors.

`settlement-window-model.py` reports 55 assertions over PREACTIVE/migration, early-return semantics, force-due precedence, renewable recovery, no-seat escape, triple-bounded prefixes, one-anchor chronology, sealed data sessions, frozen admission epochs, replay and timing geometry. Cryptographic booleans in that model are not cryptographic evidence.

`commitment-model.py` reports 12 vectors/properties for the exact EIP-712, registry, forced queue, session MMR, recovery ID, KZG transcript/precompile, body and blob encodings. The conformance and proof gates remain mandatory.

E Security invariants checklist

1. Canonical state changes only after one valid proof and only from its stored base tuple.
2. A sync transition cannot be rolled back by validation of the same call's candidate.
3. No payment, seat or standby is required for canonical commit.
4. Recovery candidates bind one live episode/revision, the current base tuple, one stored prior-block anchor at required depth and one frozen queue target. Expired rounds are permissionlessly renewable without another burn.
5. A due forced head advances in the next canonical block; appends omitted by a frozen recovery round cannot become due until that round expires. Prefix count, bytes and accounted gas are independently bounded.
6. Tier 3 has no builder signature, discretionary transaction, arbitrary coinbase or blob body.
7. Every tier-1/2 block has a valid scheduled signature under the frozen admission version, strict slot progress and the candidate's one authenticated anchor no later than its L2 time.
8. Every discretionary body byte is covered exactly once by a unique, unexpired authenticated data record.
9. Schedule inputs share one authenticated execution payload and use exact Keccak encodings.
10. Tombstoned keys cannot re-enter; each window has an independent slashable tranche and finite release state.
11. Candidate, window, data-session, queue count/bytes/gas, history and storage-retention work is bounded by consensus constants and safe eviction.
12. L1 reorg replay removes canonical state, penalties and credits from orphaned transactions together.
13. Migration freezes old ingress and imports canonical/bridge cursors into PREACTIVE settlement atomically while using one shared forced queue.